Thread Test Assignment

Name: Md Mahmudur Rahman

ID : IT 24024

```
import java.util.concurrent.atomic.AtomicLong;

public class Mahmud_Thread implements
Runnable {
static final AtomicLong safeStaticCounter =
new AtomicLong (0);
static long unsafeStaticCounter =0;
private Long instanceCounter =0;
private final long increments;
private final boolean threadSafe;
public Mahmud_Thread (long increments,
boolean threadSafe) {
    this.increments = increments;
    this.threadSafe= threadSafe;
}
```

```
@ Override
public void run() {
  for (long (i=0; i< increments; i++) {
  if (threadsafe) {
    safeStaticCounter.incrementAndGet();
  } else {
    unsafeStaticCounter ++;
  }
    instanceCounter ++;
  }
}

public static void main (String[] args)
throws InterruptedException {
  if (args.length != 3) {
System.out.println ("usage:=java Mahmud_
Thread4 <thread> <increments> <threadfalse>");
```

```
return;
}
int threads;
long increments;
boolean threadSafe;
try {
threads = Integer.parseInt(args[0]);
increments = Long.parseLong(args[1]);
threadSafe = Boolean.parseBoolean(args[2]);
} catch (NumberFormatException e) {
System.out.println("Error: threads and
increments must be integers!");
return;
}
if (threads < 1 || increments < 0){
System.out.println("Error: threads must
be >= 1 and increments >= 0.");
```

```
return;
}
safeStaticCounter.set(0);
unsafeStaticCounter = 0;
Mahmud_Thread[] tasks = new Mahmud_
Thread[threads];
Thread[] workers = new Thread[threads]
for (int i=0; i< threads ; i++) {
    tasks[i] = new Mahmud_Thread (
    increments, threadsafe);
        workers[i] = new Thread(tasks[i],
        "worker-" + (i+1));
}
for (Thread t: workers) t.start();
for (Thread t : workers) t.join();
```

```
Long expected = (long) threads *
    increments;
Long staticCount = threadSafe ? SafeStaticCounter.getC(): UnsafeStatic
Counter.counter;
long nonStaticTotal = 0;
for (MahmudThread t : tasks) nonStatic
Total += t.instanceCounter;
long absDiff = Math.abs(staticCount -
nonStaticTotal);
```

71

Compile and Run

javac Mahmud_Thread.java

Java Mahmud_Thread 10 100000 True

Mode : Thread-safe (AtomicLong)

Threads (N) : 10

Increments / thread (M) : 100000

Expected count : 1000000

Static count : 1000000

Non-Static Total : 1000000

Absolute difference : 0

Percentage Difference : 0.0000%.